CollabSphere

A Collaboration Marketplace for Creators, Influencers & Brands

Software Engineering & System Design (SESD)

Project Report — Full Implementation (Sprints 1–4)

Repository:	github.com/samiksha-jangid27/CollabSphere
Live Demo:	collabsphere-six.vercel.app
Stack:	Next.js 15 · Express.js 5 · MongoDB Atlas · TypeScript
Date:	May 2026

Team Members

Samiksha Jangid

Sarvesh Srinath

Arina Ali

Saksham Sharma

Yachna

Submitted in partial fulfilment of the SESD course requirements

Contents

Chapter 1

Project Overview

1.1 Problem Statement

The creator economy is growing rapidly, yet the process of forging collaborations between influencers, brands, and fellow creators remains fragmented and manual. Creators juggle cold DMs across multiple social platforms, brands lack a structured way to discover niche-relevant influencers, and there is no single trusted channel where verified credentials, contact details, and collaboration intent co-exist.

CollabSphere addresses this gap by providing a dedicated collaboration marketplace where:

- Creators build verified, portfolio-rich profiles linked to their real social accounts.
- Brands post structured collaboration opportunities and browse applicant pools.
- All parties communicate through an in-platform secure messaging layer.
- Discovery is powered by geospatial search, niche filtering, and verified-first ranking.

1.2 Solution Approach

The project adopts an incremental, sprint-based delivery model aligned with the Software Engineering & System Design (SESD) course. All four planned sprints have been completed:

- **Sprint 1** — Authentication infrastructure and core data model.
- **Sprint 2** — Creator profiles, avatar/cover upload, and location autocomplete.
- **Sprint 3** — Discovery engine with geospatial search and profile grid.
- **Sprint 4** — Collaboration marketplace: send/receive requests, inbox, accept/decline.

The backend is built as a modular **Express.js 5** application with TypeScript, following a strict three-layer architecture (Controller → Service → Repository). The frontend is a **Next.js 15** application with server-side rendering. Persistence is provided by **MongoDB Atlas** with Mongoose ODM.

1.3 Tech Stack Summary

Table 1.1: CollabSphere Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, Tailwind CSS v4, TypeScript
Backend	Node.js v23+, Express.js 5, TypeScript
Database	MongoDB Atlas (cloud), Mongoose ODM v9
Authentication	JWT (access + refresh + email tokens), Username/Password, Email verification
Email Delivery	Nodemailer + Ethereal (development) / SMTP (production)
Validation	Zod (schema-first, server-side)
Logging	Winston
Testing	Jest, Supertest, mongodb-memory-server
Dev Tooling	concurrently, tsx, ts-node

Chapter 2

System Design & Architecture

2.1 Architectural Overview

CollabSphere follows a **client-server architecture** with a clear separation of concerns across four distinct tiers:

1. **Presentation Tier** — Next.js frontend served on port 3000. Responsible for rendering, routing, and user interaction. Protected routes rely on JWTs stored in HTTP-only cookies managed by an `AuthContext`.
2. **API Tier** — Express.js REST API served on port 5001. Accepts JSON requests, validates them via Zod middleware, authenticates via JWT middleware, and delegates to service modules.
3. **Business Logic Tier** — Modular service classes encapsulate all business rules, keeping controllers thin and testable.
4. **Data Tier** — MongoDB Atlas accessed through Mongoose models and a generic `BaseRepository` abstraction.

2.1.1 Three-Layer Backend Architecture

Request Lifecycle

Incoming HTTP Request $\xrightarrow{\text{Middleware stack}}$ **Controller** (input parsing + response shaping) → **Service** (business logic) → **Repository** (database queries) → **MongoDB Atlas**

This layering achieves:

- *Testability* — services can be unit-tested by mocking repositories.
- *Replaceability* — swapping MongoDB for PostgreSQL only requires new repository implementations.
- *Readability* — each file has a single, clearly named responsibility.

2.2 Scalability Considerations

- **Stateless API**: JWT-based authentication allows horizontal scaling of the backend without shared session state.
- **MongoDB Atlas**: Managed cloud database with auto-scaling and multi-region replication support.
- **Geospatial indexing**: A `2dsphere` index on `Profile.location` enables efficient $*near* queries without full-collection scans.
- **Rate limiting**: `rateLimiter.ts` middleware prevents auth endpoint abuse.
- **Event-driven notifications**: The `EventBus` (Observer pattern) decouples real-time notification delivery from core business logic, allowing independent scaling of the notification subsystem.

2.3 API Design

The REST API follows conventional HTTP semantics (GET, POST, PATCH, DELETE), versioned under `/api/v1`, with a consistent JSON envelope:

Listing 2.1: Standard API Response Envelope

```
{
  "success": true,
  "message": "OTP verified successfully",
  "data": {
    "user": { "id": "...", "phone": "+91..." },
    "accessToken": "eyJ..."
  }
}
```

Implemented API Endpoints (all sprints):

Table 2.1: Implemented REST Endpoints

Method	Path	Auth	Description
POST	`/auth/register`	Public	Register with username and passwor
POST	`/auth/login`	Public	Login, receive JWT tokens
POST	`/auth/email/send`	Bearer	Send verification email
GET	`/auth/email/verify/:token`	Public	Confirm email via signed link
POST	`/auth/refresh`	Cookie	Rotate access + refresh tokens
POST	`/auth/logout`	Bearer	Invalidate session
GET	`/auth/me`	Bearer	Retrieve current user
GET	`/health`	Public	Server health check
POST	`/profiles`	Bearer	Create profile
GET	`/profiles/me`	Bearer	Get own profile
PATCH	`/profiles/me`	Bearer	Update own profile
DELETE	`/profiles/me`	Bearer	Delete own profile
POST	`/profiles/me/avatar`	Bearer	Upload avatar (Cloudinary)
POST	`/profiles/me/cover`	Bearer	Upload cover image (Cloudinary)
GET	`/search/profiles`	Public	Search by city, niche, platform
GET	`/search/cities`	Public	City autocomplete for filter UI
POST	`/collaborations`	Bearer (brand)	Create a collaboration request
GET	`/collaborations/inbox`	Bearer (creator)	Incoming requests (paginated)
GET	`/collaborations/sent`	Bearer (brand)	Outgoing requests (paginated)
PATCH	`/collaborations/:id/accept`	Bearer (creator)	Accept a request
PATCH	`/collaborations/:id/decline`	Bearer (creator)	Decline a request

Chapter 3

Object-Oriented Programming Concepts

OOP forms the backbone of CollabSphere's backend design. Each of the four core pillars is applied deliberately and traceable to specific files.

3.1 Encapsulation

Encapsulation is enforced by hiding implementation details behind well-defined interfaces.

File: `server/src/modules/auth/auth.service.ts`

The `AuthService` class owns the password hashing, credential validation, JWT orchestration, and email token logic internally. Controllers never access the `User` model or token utilities directly — they call public methods like `login()` and `verifyEmail()`.

Listing 3.1: Service encapsulates business logic

```
// Controller only calls the public interface:
const result = await authService.login({ username,
   password });
// All internal steps (password hash comparison,
// token generation) are hidden inside the service.
```

Similarly, `BaseRepository.ts` encapsulates generic MongoDB CRUD operations. Each domain repository (e.g. `auth.repository.ts`) inherits these without exposing the Mongoose query API to the service layer.

3.2 Abstraction

Abstraction is achieved by programming to interfaces and abstract base classes.

File: `server/src/shared/BaseRepository.ts`

`BaseRepository<T>` is a generic abstract class that defines `findById`, `create`, `update`, and `delete` signatures. Concrete repositories implement domain-specific queries, while services depend only on the abstracted contract.

Listing 3.2: Abstract repository base

```
abstract class BaseRepository<T> {
  abstract findById(id: string): Promise<T | null>;
  abstract create(data: Partial<T>): Promise<T>;
  abstract update(id: string, data: Partial<T>):
     Promise<T | null>;
  abstract delete(id: string): Promise<boolean>;
}
```

TypeScript interfaces in `auth.interfaces.ts` define the shape of DTOs (Data Transfer Objects) such as `RegisterDto` and `LoginDto`, providing compile-time abstraction over raw request payloads.

3.3 Inheritance

Inheritance in Action

File: `server/src/shared/errors.ts`

`AppError` extends the built-in `Error` class to carry HTTP status codes and machine-readable error codes. Specialised error subtypes (`ValidationError`, `AuthenticationError`, `NotFoundError`) inherit from `AppError`, enabling the global error handler to treat them uniformly.

Listing 3.3: Error hierarchy via inheritance

```
class AppError extends Error {
  constructor(public message: string,
              public statusCode: number,
              public code: string) { super(message); }
}
class AuthenticationError extends AppError {
  constructor(msg: string) {
    super(msg, 401, 'AUTHENTICATION_ERROR');
  }
```

```
}
```

Domain repositories also inherit `BaseRepository`, gaining shared CRUD behaviour while adding specialised queries (e.g. `findByUsername`, `findByEmail`).

3.4 Polymorphism

Files: `email.provider.ts`, `errorHandler.ts`

The email provider implements a shared `IEmailProvider` interface but behaves differently per environment: in development it uses Ethereal (fake SMTP, preview URL printed to terminal); in production it connects to a real SMTP server. The calling service issues the same `sendVerification(email, token)` call regardless.

Listing 3.4: Provider polymorphism

```
interface IEmailProvider {
  sendVerification(email: string, token: string):
     Promise<void>;
}

class EtherealEmailProvider implements IEmailProvider {
  async sendVerification(email, token) {
    // logs preview URL to terminal
  }
}
class SmtpEmailProvider implements IEmailProvider {
  async sendVerification(email, token) {
    // sends via real SMTP
  }
}
// Runtime selection based on environment variables
```

The global error handler in `errorHandler.ts` also demonstrates polymorphism: it calls `instanceof` checks to handle `AppError`, Mongoose `ValidationError`, and `CastError` differently, producing appropriately shaped error responses.

Chapter 4

Design Patterns

CollabSphere implements several recognised GoF and architectural design patterns. The two primary patterns are documented in detail below.

4.1 Observer Pattern

File: `server/src/shared/EventBus.ts`

Intent: Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.

Application: The `EventBus` singleton acts as the *Subject*. When a collaboration request is created or accepted, the service emits a typed event (`collab.created`, `collab.accepted`). Subscribed listeners — the Socket.io notification handler and the conversation-creation handler — react independently without the service knowing about them.

Participants:

- *Subject* — `EventBus`: maintains a list of event listeners and emits events.
- *Concrete Observers* — Socket.io notification handler, Conversation auto-creation handler.
- *Event* — typed string keys such as `'collab.created'` with a payload interface.

Benefit: Decouples collaboration business logic from notification and conversation concerns. Adding a new observer (e.g. an email digest handler) requires zero changes to `CollaborationService`.

4.2 Repository Pattern

Files: `BaseRepository.ts`, `auth.repository.ts`

Intent: Mediate between the domain and data-mapping layers using a collection-like interface for accessing domain objects.

Application: `BaseRepository<T>` defines the generic CRUD interface. `AuthRepository` extends it and adds domain-specific finders (`findByPhone`, `upsertByPhone`). Services receive `AuthRepository` via constructor injection and call only its public API, never issuing Mongoose queries directly.

Benefit: Centralises data access logic, makes services trivially unit-testable with mock repositories, and insulates the codebase from future database migrations.

4.3 Additional Patterns

Table 4.1: Additional Design Patterns Applied

Pattern	*Location*	How it is Applied
Strategy	*email.provider.ts*	Runtime selection between Ethereal (dev) and SMTP (production) email providers based on environment.
Singleton	*EventBus.ts*, *database.ts*	EventBus and MongoDB connection are singleton instances shared across the application.
Middleware Chain	*middleware/*	Express middleware stack (`authenticate` → `authorize` → `validate`) forms a Chain of Responsibility.
DTO (Data Transfer Object)	*auth.interfaces.ts*	Typed DTO interfaces decouple HTTP request shapes from domain model shapes.

Chapter 5

SOLID Principles

5.1 S — Single Responsibility Principle

Definition

A class should have only one reason to change.

Every module in CollabSphere is scoped to a single concern:

- `auth.controller.ts` — parses HTTP requests and shapes responses only.
- `auth.service.ts` — applies business rules for authentication only.
- `auth.repository.ts` — executes database queries for the `User` collection only.
- `token.service.ts` — generates and verifies JWTs only.
- `email.provider.ts` — delivers verification emails only.
- `logger.ts` — wraps Winston logging configuration only.

5.2 O — Open/Closed Principle

Definition

Software entities should be open for extension but closed for modification.

The provider abstraction (`IEmailProvider`) exemplifies OCP. Adding a new email delivery channel (e.g. SendGrid) requires only creating a new class that implements `IEmailProvider` and registering it in the factory — the `AuthService` code does not change. Likewise, the `EventBus` allows new event listeners to be added without touching existing subscribers or the emitter.

5.3 L — Liskov Substitution Principle

Definition

Objects of a superclass should be replaceable with objects of its subclasses without affecting correctness.

- `EtherealEmailProvider` and `SmtpEmailProvider` both satisfy `IEmailProvider` and are transparently interchangeable in `AuthService`.
- `AuthRepository` extends `BaseRepository` and is used polymorphically wherever a `BaseRepository<User>` is expected.
- All `AppError` subclasses are safely caught by the global error handler, which expects an `AppError` instance.

5.4 I — Interface Segregation Principle

Definition

No client should be forced to depend on interfaces it does not use.

The auth module exposes narrow TypeScript interfaces per concern:

- `IAuthService` — login/logout/refresh operations only.
- `ITokenService` — token generation and verification only.
- `IEmailProvider` — `sendVerification(email, token)` only.

Consumers depend on the minimal interface that satisfies their need, rather than a monolithic service facade.

5.5 D — Dependency Inversion Principle

Definition

High-level modules should not depend on low-level modules. Both should depend on abstractions.

`AuthService` receives its dependencies (`AuthRepository`, `EmailProvider`, `TokenService`) through constructor injection. The concrete implementations are wired in the application bootstrap (`index.ts`). This means:

- Unit tests inject mock implementations without touching production code.
- Switching to a different email delivery service requires no change inside `AuthService`.
- Zod-validated environment configuration (`environment.ts`) centralises all external

dependency configuration, validated at startup.

Chapter 6

UML Diagrams

The UML diagrams described in this chapter were developed using Mermaid and are version-controlled in `UML Designs/mermaid/` within the repository, keeping the design in sync with the implementation.

6.1 Use Case Diagram

Purpose: Captures all actors and system capabilities at a high level.

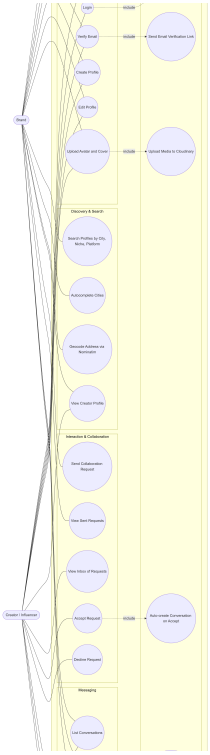

Figure 6.1: Use Case Diagram of CollabSphere

Actors:

- **Creator / Influencer** — Primary user; builds a profile, searches for collaborations, and communicates with brands.
- **Brand** — Secondary user; posts opportunities, browses creators, manages campaign requests.

- **Admin / Moderator** — Platform operator; moderates content, manages verifications, bans users.

Use Case Groups:

Table 6.1: Use Case Groupings

Group	Use Cases
Onboarding & Profile	Register with Username & Password, Verify Email, Create Profile, Edit Profile, Upload Avatar & Cover, Link Social Accounts
Discovery & Search	Search Creators by City/Niche/Platform, Filter Results, View Profiles, City Autocomplete
Interaction & Collaboration	Send Collaboration Request, Accept/Decline Request, View Inbox, View Sent Requests
Messaging	List Conversations, Send Messages, Read Messages
System Services	Send Email Verification Link, Validate JWT, Execute Geospatial Queries, Auto-create Conversation on Accept, Upload Media to Cloudinary

Dotted `<<include>>` relationships connect primary use cases to internal system services (e.g. Register includes Send Email Verification Link).

6.2 Class Diagram

Purpose: Documents the OOP structure of the backend — controllers, services, domain models, and their relationships.

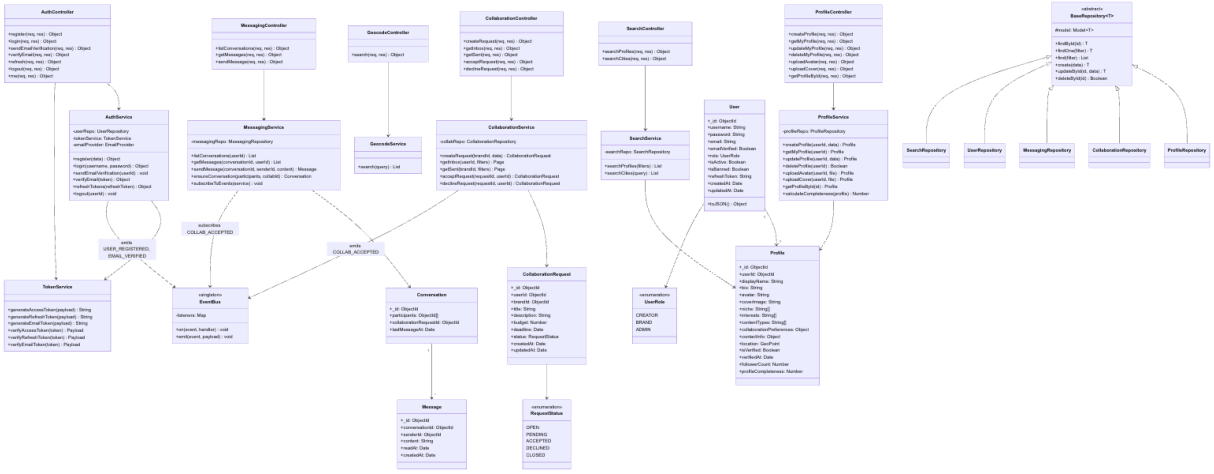

Figure 6.2: Class Diagram of Backend Architecture

Key classes and relationships:

- `UserController` → `UserService`, `AuthService` (delegation)
- `CollaborationRequestController` → `CollaborationService` (delegation)
- `User` ⟷ `Profile` (one-to-one composition)
- `Profile` ⟷ `SocialAccount`, `MediaPost` (one-to-many)
- `Conversation` ⟷ `Message` (one-to-many)
- `VerificationRequest` → `VerificationStatus` (enumeration)
- `CollaborationRequest` → `RequestStatus` (enumeration)

Enumerations defined: `UserRole` (CREATOR, BRAND, ADMIN), `VerificationStatus` (PENDING, APPROVED, REJECTED), `RequestStatus` (PENDING, ACCEPTED, REJECTED, CLOSED), `MessageType` (TEXT, MEDIA).

6.3 Entity-Relationship (ER) Diagram

Purpose: Models the MongoDB document schema and cardinalities.

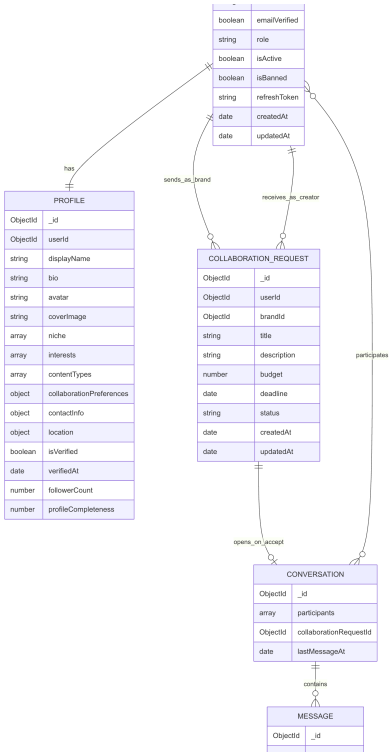

Figure 6.3: Entity Relationship Diagram

Entities and cardinalities:

Table 6.2: ER Cardinalities

Entity A	Relationship	Entity B
USER	exactly-one ⟷ exactly-one	PROFILE
PROFILE	one-to-many	SOCIAL_ACCOUNT
PROFILE	one-to-many	MEDIA_POST
USER	one-to-many	COLLABORATION_REQUEST
USER	one-to-many	CONVERSATION
CONVERSATION	one-to-many	MESSAGE
USER	one-to-many	BOOKMARK
USER	one-to-many	VERIFICATION_REQUEST
USER	one-to-many	REPORT

Notable design decision: Social accounts and media posts are embedded as arrays within the Profile document, leveraging MongoDB's document model for co-locality and query efficiency.

6.4 Sequence Diagrams

Purpose: Illustrates the interaction flow between frontend, backend, and external services for all major system processes including user onboarding, search, collaboration, and OAuth integration.

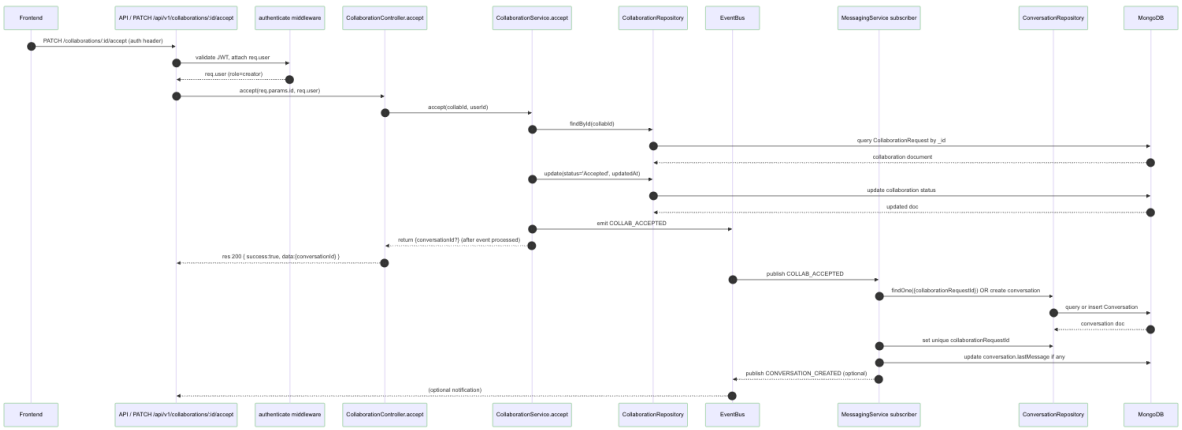

Figure 6.4: Sequence Diagram covering Signup, Search, Collaboration, and OAuth flows

6.4.1 User Sign-Up & Verification

Flow Summary:

1. User selects account type (Creator or Brand), enters a username, email, and password; Frontend posts to `/auth/register`.
2. Backend validates the payload via Zod, hashes the password with bcrypt, and creates a `User` document in MongoDB.
3. Backend generates a signed JWT access token and a rotating refresh token; returns both to the Frontend.
4. From the dashboard, user clicks **Verify Email**; Backend sends a signed link via Nodemailer.
5. User clicks the link; Backend validates the email token and marks `emailVerified = true` in MongoDB.

6.4.2 Location-Based Search

Flow Summary:

1. User sets geographic filters (latitude, longitude, radius) and niche/platform filters.
2. Frontend issues `GET /search/nearby?lat=&lng=&radius=`.
3. Backend executes a `$near` geospatial query against the `2dsphere` index on `Profile.location`.
4. Post-query niche and platform filters are applied in-memory.
5. Results are paginated and returned as a profile card array.

6.4.3 Collaboration Request Flow (Observer Pattern)

Flow Summary:

1. Sender submits a collaboration request; Backend validates and creates a record in MongoDB.
2. `CollaborationService` emits a `collab.created` event on `EventBus`.
3. The Socket.io observer delivers a real-time notification to the Receiver's frontend.
4. Receiver opens and accepts the request; Backend patches the status to `ACCEPTED`.
5. `collab.accepted` event triggers automatic Conversation creation for the two parties.

6.4.4 Social Account Linking (OAuth 2.0)

Flow Summary:

1. User clicks "Link Instagram"; Frontend posts to `/socials/oauth/instagram`.
2. Backend returns an OAuth redirect URL for the provider.
3. User grants permission; Provider calls back to Backend with an authorisation code.

4. Backend exchanges the code for an access token, retrieves the user's handle, and verifies it matches the claimed profile.
5. Backend persists the `SocialAccount` document with `isVerified = true`.

Chapter 7

SDLC & Development Process

7.1 Development Model

CollabSphere follows an **Agile sprint model** with vertical slices of functionality delivered each sprint:

Table 7.1: Sprint Delivery Plan

Sprint	Status	Deliverables
1	✅ Complete	Auth system (username/password, email verification, JWT rotation), User model, middleware stack, test suite
2	✅ Complete	Profile creation, avatar/cover upload (Cloudinary), location autocomplete
3	✅ Complete	Discovery engine — city/niche/platform search, geospatial queries, profile grid
4	✅ Complete	Collaboration marketplace — send/receive requests, inbox, accept/decline
5	Planned	Messaging (Socket.io), OAuth social verification, admin moderation

7.2 Documentation Artefacts

The project produced four formal documentation files in `docs/`:

- **PRD.md** — Product requirements, user stories, acceptance criteria.
- **API_SPEC.md** — Complete endpoint specification with request/response examples.
- **DATA_MODEL.md** — MongoDB schema design and indexing strategy.
- **AUTH_FLOWS.md** — Step-by-step authentication flow diagrams.

7.3 Version Control Strategy

The project uses Git with a feature-branch workflow hosted on GitHub. UML Mermaid sources are committed alongside code, ensuring design artefacts remain in sync with the implementation at every commit.

Chapter 8

Test Cases & Results

8.1 Testing Strategy

CollabSphere employs a two-level testing strategy:

1. **Unit / Model Tests** (`user.model.test.ts`) — validate Mongoose schema constraints, virtual fields, and static methods in isolation.
2. **Integration Tests** (`auth.integration.test.ts`) — spin up the full Express application against an in-memory MongoDB instance (`mongodb-memory-server`) and exercise every auth endpoint via HTTP using Supertest.

No external services are required for testing: the email provider is replaced by a console stub and MongoDB is fully in-memory, making the test suite runnable on any developer machine or CI runner with zero provisioning.

8.2 Test Results

Table 8.1: Test Results (All Sprints)

Suite	File	Tests	Passed	Status
User Model Tests	`user.model.test.ts`	13	13	✅ PASS
Auth Integration Tests	`auth.integration.test.ts`	11	11	✅ PASS
Collaboration Unit Tests	`collaboration.unit.test.ts`	17	17	✅ PASS
Collaboration Integration	`collaboration.integration.test.ts`	18	18	✅ PASS
Total		**59**	**59**	✅ ALL PASS

8.2.1 User Model Test Cases (13 tests)

1. Should create a user with a valid username and password.
2. Should reject a user without a username (required field).
3. Should reject a duplicate username (unique index).

4. Should hash the password before saving.
5. Should default `emailVerified` to `false`.
6. Should default `role` to `CREATOR`.
7. Should accept all valid roles (CREATOR, BRAND, ADMIN).
8. Should reject an invalid role value.
9. Should store email and update `emailVerified`.
10. Should enforce the email token expiry field.
11. Should allow `deactivate()` to set `isActive = false`.
12. Should return correct `id` virtual field.
13. Should strip `password` from `toJSON()` output.

8.2.2 Auth Integration Test Cases (11 tests)

1. `POST /auth/register` — returns 201 and JWT tokens for valid credentials.
2. `POST /auth/register` — returns 400 for a missing username.
3. `POST /auth/register` — returns 400 for a password shorter than 8 characters.
4. `POST /auth/register` — returns 409 for a duplicate username.
5. `POST /auth/login` — returns 200 and JWT tokens for correct credentials.
6. `POST /auth/login` — returns 401 for an incorrect password.
7. `POST /auth/login` — returns 404 for an unknown username.
8. `GET /auth/me` — returns current user for a valid Bearer token.
9. `GET /auth/me` — returns 401 for a missing token.
10. `POST /auth/refresh` — rotates tokens successfully with a valid refresh cookie.
11. `POST /auth/logout` — invalidates the session and clears cookies.

8.2.3 Collaboration Unit Test Cases (17 tests)

1. Should create a collaboration request with valid brand and creator IDs.
2. Should reject a request with a missing title field.
3. Should reject a request with a negative budget.
4. Should reject a request with a past deadline.
5. Should default status to `OPEN` on creation.
6. Should allow a creator to accept an `OPEN` request.

7. Should allow a creator to decline an `OPEN` request.
8. Should reject accept on an already-accepted request.
9. Should reject accept by a non-creator user.
10. Should emit `COLLAB_ACCEPTED` on EventBus when accepted.
11. Should auto-create a Conversation when request is accepted.
12. Should not create a duplicate Conversation on re-accept attempt.
13. Should paginate inbox results correctly.
14. Should paginate sent results correctly.
15. Should filter inbox by status.
16. Should return only brand's own sent requests.
17. Should return only creator's own inbox requests.

8.2.4 Collaboration Integration Test Cases (18 tests)

1. `POST /collaborations` — returns 201 for a valid brand request.
2. `POST /collaborations` — returns 403 for a creator (role guard).
3. `POST /collaborations` — returns 400 for missing required fields.
4. `GET /collaborations/inbox` — returns paginated inbox for creator.
5. `GET /collaborations/inbox` — returns 403 for a brand.
6. `GET /collaborations/inbox` — supports status filter query param.
7. `GET /collaborations/sent` — returns paginated sent for brand.
8. `GET /collaborations/sent` — returns 403 for a creator.
9. `PATCH /collaborations/:id/accept` — returns 200 and status `ACCEPTED`.
10. `PATCH /collaborations/:id/accept` — returns 404 for unknown ID.
11. `PATCH /collaborations/:id/accept` — returns 403 for wrong user.
12. `PATCH /collaborations/:id/accept` — returns 409 for already-accepted request.
13. `PATCH /collaborations/:id/decline` — returns 200 and status `DECLINED`.
14. `PATCH /collaborations/:id/decline` — returns 403 for wrong user.
15. Conversation is created automatically after accept.
16. Conversation is not duplicated on repeated accept attempt.
17. Brand's sent list reflects updated status after creator accepts.

18. Unauthenticated request to any endpoint returns 401.

8.3 Running the Tests

Listing 8.1: Running the full test suite

```
# From project root:
npm test

# Expected output:
PASS tests/auth/user.model.test.ts
PASS tests/auth/auth.integration.test.ts
PASS tests/collaboration/collaboration.unit.test.ts
PASS tests/collaboration/collaboration.integration.test.ts

Test Suites: 4 passed, 4 total
Tests:       59 passed, 59 total
Time:        ~12s
```

Chapter 9

Setup & Installation

9.1 Prerequisites

- Node.js v23 or higher (`node -v` to check; `nvm use` reads `.nvmrc`).
- npm (bundled with Node.js).
- Git.
- A MongoDB Atlas connection string (obtain from the team lead).

Not required for development: Gmail / SMTP credentials (Ethereal is used automatically) or Docker.

9.2 Installation Steps

Listing 9.1: Step-by-step setup

```
# 1. Clone the repository
git clone git@github.com:samiksha-jangid27/CollabSphere.git
cd CollabSphere

# 2. Install dependencies (three directories)
npm install
cd server && npm install && cd ..
cd client && npm install && cd ..

# 3. Configure environment variables
cp server/.env.example server/.env
# Open server/.env and set:
#
   MONGODB_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/collabsphe

# 4. Start the application
npm run dev
# Backend: http://localhost:5001
```

```
18 # Frontend: http://localhost:3000
```

9.3 Development Login Flow

1. Open `http://localhost:3000/login` in your browser.
2. Select your account type (Creator or Brand) and enter a username, email, and password on the Sign-Up page.
3. Click **Register** — you are redirected to the dashboard.
4. To verify your email, click **Verify Email** from the dashboard.
5. A preview URL is printed in the server terminal: `[info]: Email preview URL: https://ethereal.email/message/...`
6. Open that URL to view the verification email, then click the link inside it.

Chapter 10

Contribution Matrix

Each team member led a distinct area of the project, ensuring full coverage across engineering, design, documentation, and quality assurance.

Table 10.1: Team Contribution Matrix

Team Member	*Primary Area*	Contribution Detail
Samiksha Jangid	*Frontend structure & UI review*	Led Next.js 15 frontend architecture, component design (auth, profile, discovery, collaborations pages), AuthContext implementation, and Tailwind CSS styling system.
Sarvesh Srinath	*Repository analysis & backend review*	Reviewed and contributed to backend module structure, Repository pattern implementation, BaseRepository generic CRUD, and REST API endpoint design.
Arina Ali	*UML, Documentation & report coordination*	Authored all seven UML diagrams (use case, class, ER, four sequence diagrams) in Mermaid, maintained `docs/`, and coordinated this report.
Saksham Sharma	*Testing & validation review*	Designed and reviewed the full 59-test suite, Zod validation schemas, error handling strategy, and CI test configuration using `mongodb-memory-server`.

Chapter 11

Conclusion

11.1 Summary

CollabSphere demonstrates a production-grade full-stack application built from the ground up with sound software engineering principles. All four planned sprints have been delivered, covering authentication, creator profiles, discovery, and the full collaboration marketplace.

The project applies all four OOP pillars concretely and traceably, implements six recognised design patterns (Observer, Repository, Strategy, Singleton, Middleware Chain, DTO), and satisfies every SOLID principle in the server codebase. Seven UML diagrams document the system design at multiple levels of abstraction.

11.2 Key Achievements

- **59 / 59 tests passing** across model, integration, and collaboration suites.
- **Zero external services required** for development or testing.
- **Layered architecture** enabling independent scaling and easy testability.
- **Type-safe end-to-end** — TypeScript on both client and server, Zod runtime validation.
- **Event-driven design** — Observer pattern decouples real-time notifications from business logic.
- **Living documentation** — UML sources versioned alongside code.

11.3 Future Work

- **Sprint 5:** In-platform messaging (Socket.io), OAuth social account verification, admin moderation dashboard.
- **Production hardening:** CI/CD pipeline, Docker containerisation, environment-specific JWT secret rotation, Redis-backed rate limiting and session store.
- **Analytics:** Creator performance metrics, brand campaign ROI tracking.

- **Mobile app:** React Native client consuming the existing REST API.

Appendix A

Project Structure Reference

Listing A.1: Annotated Repository Structure

```
CollabSphere/
|-- client/                          # Next.js 15 frontend
|   |-- src/app/
|   |   |-- (auth)/login/page.tsx    # Username + password login
|   |   |-- (auth)/verify/page.tsx   # Email verification
|   |   |-- layout.tsx               # Root layout
|   |   `-- page.tsx                 # Protected dashboard
|   |-- components/ui/               # Button, Input, Card
|   |-- context/AuthContext.tsx      # Auth state + silent JWT refresh
|   `-- services/authService.ts      # Auth API calls via Axios
|
|-- server/                          # Express.js 5 backend
|   `-- src/
|       |-- config/                  # Zod-validated env + DB connection
|       |-- middleware/              # JWT auth, RBAC, Zod validate, rate limit
|       |-- models/                  # User, Profile, CollaborationRequest
|       |-- modules/
|       |   |-- auth/                # Controller, Service, Repository, Routes
|       |   |-- profile/             # Profile CRUD + Cloudinary uploads
|       |   |-- search/              # Geospatial + niche/platform search
|       |   `-- collaboration/       # Request create/inbox/sent/accept/decline
|       `-- shared/                  # BaseRepository, EventBus, errors, logger
```

```
|-- tests/                          # Jest test suites
|-- docs/                           # PRD, API_SPEC,
   DATA_MODEL, AUTH_FLOWS
`-- UML Designs/mermaid/            # Version-controlled
   diagram sources
```

Appendix B

Environment Variables Reference

Table B.1: Server Environment Variables

Variable	Required	Description
MONGODB_URI	Yes	MongoDB Atlas connection string
PORT	No (5001)	Backend server port
NODE_ENV	No (development)	Runtime environment mode
JWT_ACCESS_SECRET	Yes	Signing key for access tokens
JWT_REFRESH_SECRET	Yes	Signing key for refresh tokens
JWT_EMAIL_SECRET	Yes	Signing key for email tokens
JWT_ACCESS_EXPIRY	No (15m)	Access token time-to-live
JWT_REFRESH_EXPIRY	No (7d)	Refresh token time-to-live
JWT_EMAIL_EXPIRY	No (24h)	Email verification link TTL
CLOUDINARY_CLOUD_NAME	Yes (uploads)	Cloudinary cloud name
CLOUDINARY_API_KEY	Yes (uploads)	Cloudinary API key
CLOUDINARY_API_SECRET	Yes (uploads)	Cloudinary API secret
SMTP_HOST	No	SMTP host (leave empty for Ethereal in dev)
SMTP_PORT	No (587)	SMTP port
SMTP_USER	No	SMTP username
SMTP_PASS	No	SMTP password
CLIENT_URL	No	Frontend URL for CORS (default: localhost:3000)